

NLU course project (Natural Language Understanding part)

Gabriele Masciulli (258394)

University of Trento

`gabriele.masciulli@studenti.unitn.it`

1. AI Usage Declaration

This project utilized AI assistance for boilerplate code generation, general project structure setup, CTRAN architecture re-adaptation for the CNN and Tranformer part, and debugging throughout the implementation process. All core algorithmic design, experimental methodology, and analysis were conducted independently.

2. Introduction

This project addresses the joint tasks of intent classification and slot filling using the ATIS dataset. Two approaches are explored. In Part 2.A, a baseline LSTM model is enhanced with bidirectionality and dropout. In Part 2.B, a pre-trained BERT model [1] is fine-tuned using a multi-task architecture inspired by the CTRAN framework [2], with attention to handling sub-tokenization and enriching BERT's representations using CNN layers.

3. Implementation details

3.1. Part 2.A – LSTM Enhancements

The baseline model (Model IAS) was incrementally modified. First, LSTM layers were made **bidirectional**, allowing each token's representation to benefit from both past and future context. This was particularly beneficial for slot filling, which is sensitive to the surrounding context of each token, and also improved intent classification, which requires understanding the entire utterance.

Next, a **dropout** layer was introduced between the LSTM and the output layers. This addition helped regularize the model by reducing overfitting, especially after increasing the model's capacity (hidden dimension, number of LSTM layers and embedding size). Extensive hyperparameter tuning was carried out to balance model capacity and regularization. An experiment with a **weighted cross-entropy loss** was conducted to automatically balance the two tasks: $\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{slot}} + \beta \cdot \mathcal{L}_{\text{intent}}$, where α and β are learnable parameters initialized to 1.0. This approach was motivated by the potential for the slot tagging and intent classification tasks to have different loss scales and learning dynamics, requiring adaptive weighting rather than fixed equal weighting. However, the learnable weights led to inferior generalization performance compared to the standard summed loss ($\mathcal{L} = \mathcal{L}_{\text{slot}} + \mathcal{L}_{\text{intent}}$), so the simpler approach was retained.

3.2. Part 2.B – BERT Fine-Tuning with CNN

A preliminary analysis of ATIS utterance lengths revealed a maximum of 52 tokens as can be seen from A, which informed the choice of sequence length limits for BERT's tokenizer to ensure efficient processing without truncation.

The second part fine-tunes BERT for the joint tasks. Inspired by the CTRAN architecture, this model combines three

complementary neural components: BERT provides rich contextual embeddings, multi-scale CNN layers (with kernel sizes 1, 2, 3, 5) extract local n-gram patterns at different granularities, and a Transformer encoder models global dependencies between CNN features. This hierarchical design leverages each architecture's strengths: BERT for semantic understanding, CNNs for efficient local pattern detection, and Transformers for long-range contextual modeling.

The architecture processes input as follows: BERT generates contextual embeddings, which are fed through parallel convolutions to capture multi-scale local patterns. The concatenated CNN outputs then pass through a Transformer encoder to model global sequence interactions, creating enriched representations, specifically for the slot filling task.

From BERT's outputs, the model routes information into two distinct task-specific branches:

- For **slot filling**, BERT's sequence output flows through the CNN-Transformer pipeline: multi-scale convolutions extract local n-gram patterns, followed by a Transformer encoder that models global dependencies. This enriched representation then passes through a token-level classification head to predict BIO-style labels. To handle BERT's sub-tokenization, a selective loss masking strategy was implemented: only the first sub-token of each original word contributes to the loss calculation and evaluation metrics. These were identified using the offset mapping returned by `BertTokenizer`, where the starting character offset equals 0. Continuation sub-tokens (those beginning with "##") and special tokens are assigned an ignore label and excluded from loss computation, ensuring accurate alignment between predicted and ground truth slot labels.
- For **intent classification**, BERT's pooled output ([CLS] token) passes through a classification head to predict the final intent label. This sequence-level representation is specifically designed to capture utterance-level semantics, making it ideal for intent detection without requiring the fine-grained local pattern analysis needed for slot filling.

The model jointly optimizes a summed cross-entropy loss: one for the slot filling task (token-level) and one for intent classification (utterance-level), encouraging effective multi-task learning from the shared encoder.

4. Results

Table 1: *Evaluation on ATIS test set*

Model	Intent Acc. (%)	Slot F1 (%)
LSTM + BiDir + Dropout	94.44	92.51
	94.96	93.05
BERT + CNN	97.31	97.64 (Micro)

The BERT-based model outperforms the LSTM variants by a wide margin. While Micro Slot F1 reaches 97.64%, macro-averaged F1 is notably lower (46.58%), reflecting label imbalance. Nonetheless, the integration of CNN for local context and the handling of sub-token labels proved highly effective.

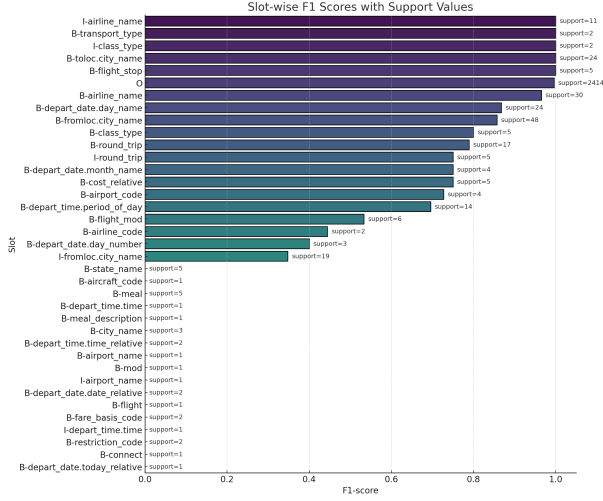


Figure 1: The slot-wise performance analysis highlights a stark imbalance in class-wise F1 scores. While frequent slots achieve high F1 scores, many rare slot types—especially those with fewer than 5 instances—suffer from extremely low or zero F1. These results suggest that the model is effective overall but struggles with underrepresented or ambiguous slot classes.

5. References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [2] M. Rafiepour and J. Salimi Sartakhti, “Ctran: Cnn-transformer-based network for natural language understanding,” *arXiv preprint arXiv:2303.10606*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.10606>

A. Part 2B: Utterance Length Analysis

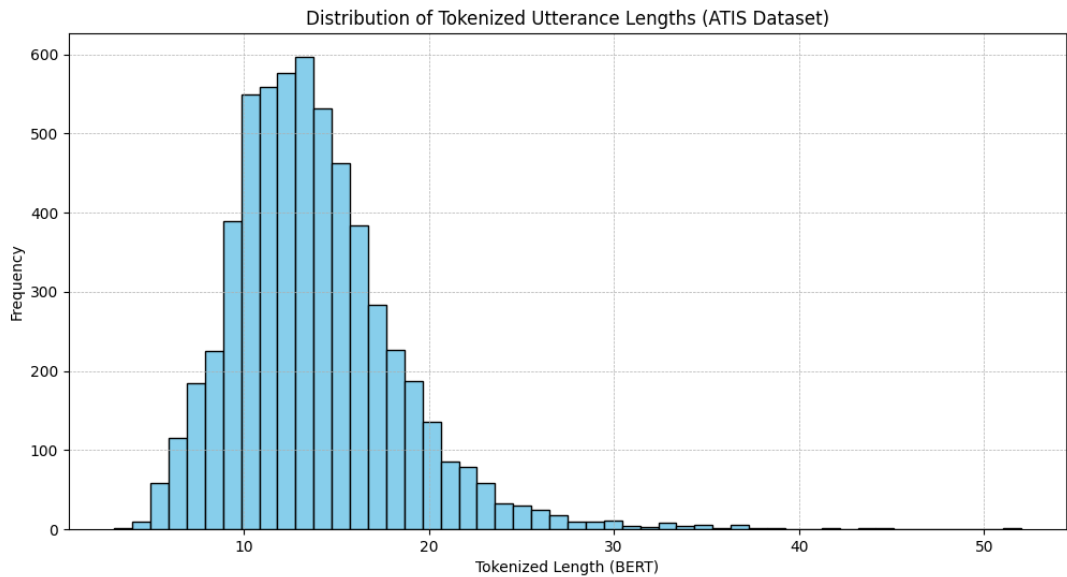


Figure 2: ATIS utterance length distribution. This analysis informed the maximum sequence length parameter for BERT tokenization i.e. 52, ensuring efficient processing without content truncation.

B. Appendix: Hyperparameter Settings for Each Model Variant

Table 2: Model Hyperparameters for Each Run

Run	Description	Hyperparameters
A1	LSTM + Bidirectional + Dropout	emb_size=200, hid_size=512, n_layers=1, lstm_dropout=0.0, fc_dropout=0.0, batch_size=128, lr=1e-4, optim=Adam
A2		emb_size=300, hid_size=500, n_layers=2, lstm_dropout=0.2, fc_dropout=0.3, batch_size=128, lr=1e-4, optim=Adam
B1	BERT + CNN	bert_model=bert-base-uncased, dropout=0.1, lr=5e-5, optim=AdamW, cnn_filters=256, (transformer).heads=8, _layers=2, _ff_dim=1024

Additional Notes:

- A-series models used evaluation batch size=64, gradient clipping=5, epochs=200 with early stopping patience=3.
- For the B model, CNN and Transformer encoder hyperparameters were initialized from CTRAN values and empirically tuned. CNN filters were increased from 128 to 256 based on preliminary experiments showing improved slot filling F1 scores. Transformer layers (2) and heads (8) were retained as they provided sufficient capacity, while further increases would require additional regularization to prevent overfitting.
- The multi-kernel CNN design ([1, 2, 3, 5]) was motivated by analysis in the CTRAN paper demonstrating that multiple kernel sizes outperform single-kernel architectures. Empirical validation confirmed that this complete kernel combination achieved superior performance compared to any subset of fewer kernels.